

Token-First Long-Term Memory for LLM Coding Agents: A Two-Budget Design, a Cognitive-Memory Lifecycle, and a Reproducible Retrieval Evaluation

Lawrence Eldridge

6 July 2026

Abstract

Retrieval-augmented memory for large-language-model coding agents is usually costed in bytes, yet the model consumes only text tokens, so byte-efficiency and token-efficiency are distinct budgets that a naive design conflates. This paper specifies *claude-ltm*, a local-first memory for Claude Code that separates the two: a detached capture path distils sessions into atomic facts, and a threshold-gated recall path injects only the facts that clear a similarity gate, at a fixed per-prompt character cap. The design is organised as CQRS with a hexagonal core so the write side (heavy, batch) and the read side (tiny, hot-path) are optimised independently, and a cognitive-memory lifecycle (a forgetting curve, rehearsal, and an offline consolidation pass) governs what survives. Retrieval quality is measured with a labelled paraphrase benchmark (64 facts, 77 queries) through the real quantised search path, across four backends. The default semantic backend, bge-base with int8 vectors, reached Recall@1 = 0.714 (95% interval [0.605, 0.803]) against a lexical stub’s 0.234, a threefold gain whose intervals do not overlap. int8 matched full-precision float exactly on every retrieval metric at roughly one-third the stored bytes. A larger model moved Recall@1 more than precision did (bge-small 0.571 to bge-base 0.714), though those intervals overlap at this sample size. These figures are honest at $n = 77$ and reported with their intervals. The design shows that model choice and what is stored, not quantisation precision, are the dominant retrieval levers at personal-store scale.

Contents

1	1 Introduction	2
1.1	1.1 Related work	2
1.2	1.2 The gap	2
1.3	1.3 The present study	3
1.4	1.4 Claims	3
2	2 Method	3
2.1	2.1 Corpus and environment	3
2.2	2.2 Variables and conditions	3
2.3	2.3 System and instruments	4
2.4	2.4 Protocol	4
3	3 Results	5
3.1	3.1 Data cleaning	5
3.2	3.2 Descriptive statistics	5
3.3	3.3 Inferential statistics	5
3.4	3.4 Reproducibility	6

4	4 Discussion	6
4.1	4.1 Hypothesis evaluation	6
4.2	4.2 Integration	6
4.3	4.3 Limitations and threats to validity	7
4.4	4.4 Implications and future directions	7
5	References	7
6	Appendix A: Raw benchmark output (this session)	8
7	Appendix B: Selected configuration defaults	8

1 Introduction

A coding agent’s context window is finite, and every token placed in it has a cost in money, latency, and displaced attention. An agent that “remembers everything” is therefore not an option: memory has to be selective, and it has to be cheap at the moment of use. The problem this paper addresses is how to give a coding agent durable, cross-session memory whose cost is paid in the currency the model actually consumes, namely text tokens, rather than in the bytes a storage-centred design optimises.

1.1 Related work

Three strands inform the design. The first is human memory research. The Atkinson and Shiffrin (1968) multi-store model separates a small, capacity-bounded short-term store from a durable long-term store, with rehearsal as the transfer mechanism; the Ebbinghaus forgetting curve describes retention decaying with time unless reinforced; Roediger and Karpicke (2006) show that retrieval itself, not only repetition, strengthens a memory (the testing effect); and the active-systems-consolidation account describes an offline pass that replays, integrates, and prunes traces. The second strand is retrieval systems: dense-vector similarity search, retrieval-augmented generation, approximate-nearest-neighbour indexes, and the quantisation of embeddings to reduce their footprint. The third is software architecture: Fowler’s Patterns of Enterprise Application Architecture and the Cosmic Python extensions, in particular Repository, Gateway with a Separated Interface, the Functional Core and Imperative Shell split, and Command Query Responsibility Segregation (CQRS).

1.2 The gap

Prior approaches leave three gaps for the coding-agent setting. First, always-on retrieval exposed as a model-callable tool carries a standing token cost and hands the model an agency decision on every turn, whether or not memory is relevant. Second, raw vector similarity recalls stale and conflicting facts, because similarity alone has no model of recency, reinforcement, or contradiction. Third, the literature optimises storage footprint, which is the wrong budget: shrinking bytes does not shrink the tokens injected into the context window, and the two can even move in opposite directions.

1.3 1.3 The present study

claude-ltm is a local-first memory and code/docs index for Claude Code, packaged as a plugin, that closes these gaps by construction. Capture runs off the interactive path and distils transcripts into atomic facts. Recall runs as a hook, not a tool, so it has zero standing token cost and no model agency; it injects only the facts that clear a similarity gate, capped at a fixed character budget. A cognitive-memory lifecycle re-ranks and retires facts by recency, reinforcement, and contradiction rather than by similarity alone. The core runs on the Python standard library, with semantic embeddings and language-model distillation as optional adapters, so the default install works with no network and no API key.

1.4 1.4 Claims

This paper advances one design claim and three measurable hypotheses.

Design claim. Treating recall as a threshold-gated hook, and treating byte-efficiency and token-efficiency as separate budgets, keeps interactive token cost near zero while retaining useful memory.

H1. int8 quantisation of the embedding vectors does not materially reduce retrieval quality relative to full-precision float, at personal-store scale.

H2. Embedding model size is a larger recall lever than vector precision: moving from a smaller to a larger model changes Recall@1 more than moving from float to int8.

H3. A larger labelled corpus narrows the confidence interval on the retrieval metrics, so a fixed between-condition difference becomes easier to resolve.

2 2 Method

2.1 2.1 Corpus and environment

Retrieval is measured with a labelled paraphrase set held in `plugins/ltm/bench/dataset.json` and executed by `plugins/ltm/bench/run_eval.py`. Each query is worded away from its target fact so that lexical overlap alone is insufficient, which stresses semantic matching. The corpus has 64 facts and 77 queries across several domains (infrastructure, data and machine learning, security, product, and process), with six multi-relevant queries and eight hard-negative facts that no query targets. It was widened from an earlier 34-fact, 29-query set during this work; the earlier set is referenced only in § 3.3 to quantify the effect of widening. The runtime is CPython 3.12 on macOS, with `fastembed` 0.8.0 providing the semantic backends. Every figure in this paper was reproduced in one session at commit `aa0bcc5`.

2.2 2.2 Variables and conditions

The independent variables are the embedding backend (`hash`, a dependency-free lexical stub, versus `fastembed` semantic models), the embedding model size (`bge-small`, 384 dimensions, versus `bge-base`, 768 dimensions), and the vector precision (int8 versus float). The dependent variables are Recall@1, Recall@3, MRR@10, and bytes per fact, with per-query latency recorded

as an operational cost. Controlled factors are the search path (the same quantised cosine scan for every quantised run), the dataset, and the caps applied during evaluation (`top_k` and `min_sim` are relaxed so ranking is measured over the full set).

2.3 2.3 System and instruments

Architecture. `claude-ltm` is CQRS with a hexagonal (ports and adapters) core. The write side and the read side have opposite performance profiles and are separated accordingly. Capture is heavy, batched, and latency-tolerant; it runs detached at the `Stop`, `SessionEnd`, and `PreCompact` hooks. Recall is tiny and latency-critical; it runs in `UserPromptSubmit` for just-in-time injection and at `SessionStart` for a stable project core. The POEAA roles are fixed: memory access is a Repository (`core/store.py`), never Active Record; the embedding provider is a Gateway behind a Separated Interface (`core/ports/embedding.py`, `core/adapters/`); distillation, ranking, scoring, and quantisation are pure functions in the Functional Core; and the injected payload is a one-line-per-fact data-transfer object rendered by `core/recall/__init__.py::render_block`, which returns the empty string on no match (a Null Object, so irrelevant turns cost nothing).

Ranking. Retrieval is a hybrid re-rank, not raw similarity. Each candidate that clears the similarity gate `min_sim` receives a priority score combining similarity, recency decay, and a frequency boost, computed by the pure `core/recall/__init__.py::_score` over `core.domain.scoring`. Superseded facts are excluded at the SQL layer, so a replaced fact cannot resurface, and the injected block is bounded by a character cap so the token budget is fixed regardless of store size.

Cognitive lifecycle. Three memory-research mechanisms are layered on the store. A forgetting curve applies exponential recency decay. Rehearsal reinforces a re-encountered fact (frequency increases, recency refreshes) instead of duplicating it, and transfers a fact from the short-term to the long-term tier once it has been rehearsed or retrieved. An offline consolidation pass (`core/consolidation/`) runs at session checkpoints and executes, in order, replay, displacement, integration, refinement, and purge; each stage is individually gated and reversible.

Storage. Each fact is stored as its text plus a quantised embedding: an int8 vector as the primary search representation and binary sign bits for a fast Hamming pre-filter. The store is a single SQLite database that survives plugin updates.

Instrument. The benchmark embeds each query, ranks the corpus through the quantised search path, and scores the position of the gold fact. Recall@k is the fraction of queries whose gold fact appears in the top k; MRR@10 is the mean reciprocal rank of the gold fact within the top ten; bytes per fact is the stored vector footprint. A `+float` run ranks on raw full-precision vectors in memory, so the gap between a backend and its `+float` twin is exactly the int8 loss.

2.4 2.4 Protocol

A captured session becomes memory by the sequence distil, embed, quantise, and persist, off the interactive path. A query becomes an injection by the sequence embed, cosine scan over the int8 vectors, similarity gate, hybrid re-rank, and top-k render under the character cap. The

benchmark runs this path for every query and aggregates the metrics. The exact command is given in the reproducibility block in § 3.4.

3 3 Results

3.1 3.1 Data cleaning

No queries were excluded, and every backend under test loaded and ran; no rows are missing. Each `fastembed` model was warmed with one throwaway embedding before timing so that the one-off model load is excluded from the per-query latency.

3.2 3.2 Descriptive statistics

All four backends were evaluated on the same 64-fact, 77-query corpus in this session.

Backend	Dim	Recall@1	Recall@3	MRR@10	Bytes/fact	Query ms
hash (lexical stub)	256	0.234	0.325	0.307	288	1.8
bge-small int8	384	0.571	0.792	0.688	432	4.7
bge-base int8 (default)	768	0.714	0.857	0.793	864	9.6
bge-base float	768	0.714	0.857	0.793	3072	7.8

3.3 3.3 Inferential statistics

The Wilson 95% intervals on Recall@1 ($n = 77$) are: hash [0.153, 0.340]; bge-small [0.460, 0.676]; bge-base [0.605, 0.803]; bge-base float [0.605, 0.803]. Three findings follow.

Semantic against lexical is robust. bge-base reaches Recall@1 = 0.714 against the lexical stub’s 0.234, a gain of 0.480 and a relative factor of about 3.0. The two intervals ([0.605, 0.803] and [0.153, 0.340]) do not overlap, so this difference is not attributable to sampling noise at this sample size.

int8 equals float. bge-base int8 and bge-base float are identical on every retrieval metric (a delta of 0.00), at 864 against 3072 bytes per fact, a footprint reduction of about 3.6 times. A zero difference needs no interval.

Model size moves the metric more than precision, but is not resolved here. The bge-small to bge-base change moves Recall@1 by 0.143, whereas the float to int8 change moves it by 0.000, so model size is the larger of the two levers. The model-size intervals ([0.460, 0.676] and [0.605, 0.803]) overlap, however, so at $n = 77$ that difference is indicative rather than conclusive; a wider corpus would be needed to resolve it. The precision null, by contrast, is exact.

3.4 3.4 Reproducibility

The figures in § 3.2 and § 3.3 were produced by

```
python3 plugins/lrm/bench/run_eval.py --backends "hash,fastembed@BAAI/bge-small-en-v1.5,fast
```

from the repository root at commit aa0bcc5, against plugins/lrm/bench/dataset.json (64 facts, 77 queries), with fastembed 0.8.0 installed. The lexical hash backend needs no dependency. The lever scenarios in Appendix A add --stm --antipatterns --integrate. Raw output is in Appendix A.

4 4 Discussion

4.1 4.1 Hypothesis evaluation

H1 (int8 approximately equals float) is supported strongly. The two conditions were identical on Recall@1, Recall@3, and MRR@10, at roughly one-third the stored bytes. The compact int8 representation is therefore kept, and a float-rescore stage is not worth building.

H2 (model size is the larger lever) is supported directionally. Model size moved Recall@1 by 0.143 while precision moved it by 0.000, so model choice is the larger lever of the two. The claim is directional rather than resolved at this sample size, because the two model conditions' intervals overlap; the precision comparison, being an exact null, is not subject to that caveat.

H3 (a larger corpus narrows the interval) is supported. Widening the corpus from 29 to 77 queries reduced the Recall@1 interval half-width from about 0.15 to about 0.09, consistent with the expected narrowing as the sample grows. This is a methodological result rather than a claim about the system: it makes the between-condition comparisons more trustworthy, and it is why the model-size difference, though still within-interval, is closer to resolution than it was before.

The design claim holds by construction rather than by a single measurement: recall is a hook with no standing token cost, the injected block is character-capped, and the empty-recall case injects nothing, so an irrelevant turn costs zero tokens.

4.2 4.2 Integration

The finding that int8 matches float sits against the vector-store convention of storing full-precision embeddings and rescoring. At personal-store scale that convention buys nothing measurable here, and the compact representation keeps a brute-force cosine scan fast (about 10 ms per query at 768 dimensions over this corpus) so that no approximate-nearest-neighbour index is needed until the store is far larger. The finding that model size is the larger lever aligns with the broader retrieval literature and relocates effort from precision, which is saturated, onto model choice and distillation quality. The cognitive-memory lifecycle is where claude-lrm departs from a plain vector store: recency decay, rehearsal, and hard supersession give the system a model of time and contradiction that similarity alone lacks.

4.3 4.3 Limitations and threats to validity

Three limitations bound these results. First, the corpus is modest at 77 queries, so the intervals are wide enough that the semantic-against-lexical gap is resolved but the model-size gap is not; the finer comparison is indicative. Second, the corpus is single-author and English-only, which limits external validity, and it exercises retrieval quality rather than the end-to-end token savings a full user study would measure. Third, the consolidation pass models transfer, integration, and forgetting in a single checkpoint rather than in a distinct sleep cycle, a deliberate engineering simplification rather than a claim of biological fidelity.

4.4 4.4 Implications and future directions

The immediate implication is that a coding-agent memory should spend its budget on model choice and distillation quality and hold vector precision at int8, which is free here. Resolving the model-size comparison calls for a larger corpus; the widening reported under H3 is a step in that direction and the harness now reports intervals for every backend on a common set.

A second line of work, staged in this repository as inert scaffolding, applies five working-memory models to the retrieval and consolidation paths: gist chunking of near-duplicate facts, an explicit split between an injected focus and a broader on-demand activated set, a long-term-working-memory retrieval scaffold, use-feedback inhibition of injected-but-unused facts, and associative spreading activation over a co-occurrence and shared-entity graph. Each ships disabled by default and is gated behind a benchmark comparison before its default is changed. These are design contributions awaiting measurement rather than validated results, and the paper reports them as such.

5 References

Source at HEAD (this repository).

- Recall read path and hybrid re-rank: `plugins/ltm/core/recall/__init__.py`.
- Capture service: `plugins/ltm/core/service.py`.
- Memory store (Repository): `plugins/ltm/core/store.py`.
- Retention score: `plugins/ltm/core/consolidation/scoring.py`.
- Benchmark harness and dataset: `plugins/ltm/bench/run_eval.py`, `plugins/ltm/bench/dataset.json`.

Prior design records.

- Two-budget model, POEAA map, and the earlier benchmark: `DESIGN.md`.
- Pattern map and layering: `.claude/rules/02-architecture/01-poeaa-and-layers.md`.

External literature.

- Atkinson, R. C., and Shiffrin, R. M. (1968). Human memory: a proposed system and its control processes.
- Ebbinghaus, H. (1885). Memory: a contribution to experimental psychology.

- Roediger, H. L., and Karpicke, J. D. (2006). Test-enhanced learning: taking memory tests improves long-term retention.
- Cowan, N. (2001). The magical number 4 in short-term memory.
- Fowler, M. (2002). Patterns of Enterprise Application Architecture.
- Percival, H., and Gregory, B. (2020). Architecture Patterns with Python.

6 Appendix A: Raw benchmark output (this session)

dataset: 64 facts, 77 paraphrased queries

backend	dim	recall@1	recall@3	mrr@10	embed_ms/fact	query
-----	---	-----	-----	-----	-----	-----
hash	256	0.234	0.325	0.307	0.179	1.83
fastembed@BAAI/bge-small-en-v1.5	384	0.571	0.792	0.688	2.404	4.702
fastembed@BAAI/bge-base-en-v1.5	768	0.714	0.857	0.793	4.743	9.60
fastembed@BAAI/bge-base-en-v1.5+float	768	0.714	0.857	0.793	4.163	7.76

95% Wilson intervals (proportion metrics):

hash (n=77):	recall@1	0.234	[0.153, 0.340]	recall@3	0.325	[0.231, 0.435]
bge-small (n=77):	recall@1	0.571	[0.460, 0.676]	recall@3	0.792	[0.689, 0.868]
bge-base int8 (n=77):	recall@1	0.714	[0.605, 0.803]	recall@3	0.857	[0.762, 0.918]
bge-base float (n=77):	recall@1	0.714	[0.605, 0.803]	recall@3	0.857	[0.762, 0.918]

Lever scenarios (hash backend), via `--stm --antipatterns --integrate`:

STM lever (stm_recall_weight):	1.0 -> stm_recall@1 0.333;	0.5 -> 0.000;	0.0 -> 0.000
Anti-pattern union (global):	recall@1 0.750	recall@3 1.000	mrr@10 0.875
Integrate (gist chunking):	6 -> 5 active facts (1 merged), recall@3 preserved 1.000		

7 Appendix B: Selected configuration defaults

Key	Default	Meaning
top_k	3	facts injected per prompt (the injected focus)
min_sim	0.12	similarity gate for injection
max_chars	800	hard character cap per injection (the token guard)
half_life_days	30	recency-decay half-life
supersede_threshold	0.85	similarity at which a newer fact retires an older one
embedding	hash	default backend; <code>fastembed</code> enables semantic recall

The full configuration surface is documented in `README.md`.